

Expression Evaluation & Conversion

Infix to Postfix

Use of Tabular Method with Following Rules

- Scan the Given Infix from **Left To Right Always**.
- If Given Infix is **without ()** then initialize stack with **#** and end with **#**
- If Given Infix is **with ()** then initialize stack **(** and end with **)**
- **+, -, *, \, %** all have **EGO** problem – they cannot sit with each other in Stack.
- **^, \$, ↑** have **NO EGO** Problem – they can sit with each other in Stack.
- Postfix answer will not have any **()**

Expression Evaluation & Conversion

Infix to Prefix

Use of Tabular Method with Following Rules

- Reverse the Given Infix Expression. (Scan the Given Infix from **Right To Left Always**)
- Find the Nearly Postfix of that reverse expression.
- If Given Infix is **without ()** then initialize stack with **#** and end with **#**
- If Given Infix is **with ()** then initialize stack **)** and end with **(**
- **+, -, *, \, %** all have NO EGO problem – they can sit with each other in Stack.
- **^, \$, ↑** have EGO Problem – they cannot sit with each other in Stack.
- Reverse the answer of Nearly Postfix [**That is our Final Answer Postfix answer will not have any ()**]

Expression Evaluation & Conversion

Postfix to Infix

Use of Stack Method with Following Rules

- Always Start with **Empty** Stack Frame.
- Scan the given expression from **Left to Right**.
- While Scanning,
 - If **operand** gets scanned then **PUSH** into the stack.
 - If **operator** gets scanned then **POP TWO** recent elements from the TOP of the stack.
- The **1st element** which get pop is **operand 2 (OP2)**
- The **2nd element** which get pop is **operand 1 (OP1)**
- Perform the operation – **OP1 <Operator> OP2**
- Now, Push the **Answer with ()** into the stack.
- Repeat the process until you finish the given expression.
- Perform only 1 operation at a time.

Expression Evaluation & Conversion

Prefix to Infix

Use of Stack Method with Following Rules

- Always Start with **Empty** Stack Frame.
- Scan the given expression from **Right to Left**.
- While Scanning,
 - If **operand** gets scanned then **PUSH** into the stack.
 - If **operator** gets scanned then **POP TWO** recent elements from the TOP of the stack.
- The **1st element** which get pop is **operand 1 (OP1)**
- The **2nd element** which get pop is **operand 2 (OP2)**
- Perform the operation – **OP1 <Operator> OP2**
- Now, Push the **Answer with ()** into the stack.
- Repeat the process until you finish the given expression.
- Perform only 1 operation at a time.

Expression Evaluation & Conversion

Prefix to Postfix

Use of Stack Method with Following Rules

- Always Start with **Empty** Stack Frame.
- Scan the given expression from **Right to Left**.
- While Scanning,
 - If **operand** gets scanned then **PUSH** into the stack.
 - If **operator** gets scanned then **POP TWO** recent elements from the TOP of the stack.
- The **1st element** which get pop is **operand 1 (OP1)**
- The **2nd element** which get pop is **operand 2 (OP2)**
- Perform the operation – **OP1 OP2 <Operator>**
- Now, Push the **Answer** into the stack.
- Repeat the process until you finish the given expression.
- Perform only 1 operation at a time.

Expression Evaluation & Conversion

Postfix to Prefix

Use of Stack Method with Following Rules

- Always Start with **Empty** Stack Frame.
- Scan the given expression from **Left to Right**.
- While Scanning,
 - If **operand** gets scanned then **PUSH** into the stack.
 - If **operator** gets scanned then **POP TWO** recent elements from the TOP of the stack.
- The **1st element** which get pop is **operand 2 (OP2)**
- The **2nd element** which get pop is **operand 1 (OP1)**
- Perform the operation – **<Operator> OP1 OP2**
- Now, Push the **Answer** into the stack.
- Repeat the process until you finish the given expression.
- Perform only 1 operation at a time.